Class 3: PCA and Dimensionality Reduction

Data-driven Systems Engineering

Agenda

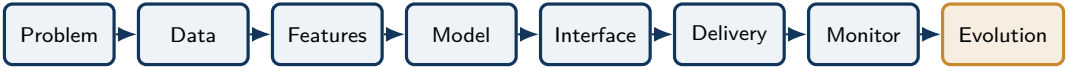
- Why high-dimensional data creates practical problems.
- PCA intuition, geometry, and workflow.
- Explained variance, interpretability, and trade-offs.
- PCA in the repository notebooks and in the fraud case.
- From notebook transformation to production-safe preprocessing.

Learning goals

Learning goals

- Interpret PCA as a geometric transformation rather than a black box.
- Decide when dimensionality reduction helps visualization, denoising, or training speed.
- Explain the trade-off between compression and interpretability.
- Connect exploratory PCA work to deployable preprocessing pipelines.

Course Map and Running Cases



Today in the course

Focus: Dimensionality reduction and compact representation.
Bridge: This class sits between raw data inspection and deliberate feature design by asking when we should compress or re-express variables.

Fraud case

In the fraud case, PCA-style transformed components show how many variables can be summarized before classification or visualization.

Pasteurization case

In the pasteurization case, correlated sensors can be projected to simpler views for interpretation, anomaly inspection, or denoising.

Course thread: The course thread moves from understanding variables to deciding which representations are useful enough to keep.

Why dimensionality reduction matters

- Many features can increase computational cost and overfitting risk.
- Highly correlated variables may carry redundant information.
- Visualization becomes difficult beyond two or three dimensions.
- Some datasets contain noise that can be filtered through projection.

PCA intuition

PCA rotates the original feature space to new orthogonal axes called principal components, ordered by the amount of variance they explain.

- The first component captures the largest possible variance.
- The second captures the largest remaining variance subject to orthogonality.
- Keeping only the leading components compresses the representation.

Typical PCA workflow

1. Standardize the features when scale differs.
2. Compute covariance structure or singular vectors.
3. Rank components by explained variance.
4. Choose how many components to retain.
5. Project the data and evaluate what was gained or lost.

Notebook example: standardize and project Iris

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import pandas as pd

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

X = StandardScaler().fit_transform(df[iris.feature_names])
pca = PCA(n_components=4)
X_pca = pca.fit_transform(X)

print(pca.explained_variance_ratio_)
```

- This mirrors ‘notebooks/Class3_PCA.ipynb‘.
- Standardization is essential because centimeters on petal and sepal features do not contribute equally by default.

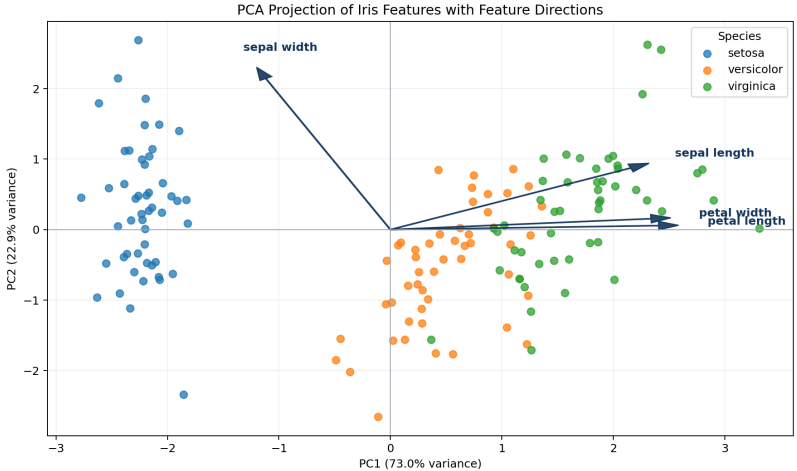
Notebook example: inspect variance and new coordinates

```
df_pca = pd.DataFrame(
    X_pca, columns=["PC1", "PC2", "PC3", "PC4"]
)
df_pca["species"] = iris.target

print(df_pca[["PC1", "PC2"]].head())
```

- PC1 and PC2 usually capture most of the separable structure in Iris.
- The projection is useful for visualization, but the new axes are weighted mixtures of original measurements.
- The notebook also uses Yellowbrick to visualize projected feature directions.

PCA projection with feature directions



Recreated from the ‘Class3_PCA.ipynb‘ Yellowbrick step with the same Iris PCA projection and feature directions.

Where PCA helps and where it hurts

Helpful when

- Visualizing structure
- Reducing collinearity
- Denoising signals
- Speeding downstream models

Risky when

- Stakeholders need direct feature meaning
- Nonlinear structure dominates
- Scaling was inconsistent
- Drift changes the meaning of the basis

Interpretability discussion

- A component is a weighted combination of original features, not a business concept by itself.
- High explained variance does not automatically imply high decision value.
- Dimensionality reduction may help the model while making explanation harder.

Fraud example

The fraud dataset already contains PCA-derived variables ‘V1‘–‘V28‘, so students should ask what interpretability remains when the original dimensions are hidden.

Repository connections

Repo activity

- ‘notebooks/Class3_PCA.ipynb‘ uses Iris to build intuition and show projection plots.
- ‘notebooks/Class3_PCA_Exercise.ipynb‘ extends the analysis to Wine.
- ‘documents/FraudDetectionAsaService.pdf‘ describes a deployed workflow where PCA-based fields are part of a real classification task.

Exercise anchor: Wine dataset for comparison

```
from sklearn.datasets import load_wine
import pandas as pd

wine = load_wine()
df_wine = pd.DataFrame(
    wine.data, columns=wine.feature_names
)
df_wine["target"] = wine.target
```

- ‘notebooks/Class3_PCA_Exercise.ipynb‘ starts here and is a good follow-up because Wine has more features and a less toy-like decision boundary.
- It lets students compare PCA as a visualization tool versus PCA as a compression step before modeling.

From notebook to production

- Fit PCA only on training data.
- Persist the scaler and PCA object together with the model.
- Apply the exact same transformation at inference time.
- Recheck the basis when the distribution of inputs changes.
- Record how many components were chosen and why.

Common mistakes

- Running PCA before understanding the meaning of the raw features.
- Comparing models trained on differently scaled inputs.
- Keeping components only because the scree plot “looks nice”.
- Forgetting that PCA can hide bias or leakage just as easily as raw features can.

Technology links

Papers and materials

scikit-learn PCA guide; PCA API reference; scikit-learn pipelines; Matplotlib documentation.

From This Class to the Next

What stays with us

- Dimensionality reduction is useful when it improves structure, visualization, or downstream learning.
- PCA is a transformation pipeline choice, not a universal preprocessing rule.
- Interpretation, scaling, and deployment consistency matter as much as variance explained.

Project use

Use PCA when you need compact representations, denoising, or lower-dimensional plots, but keep the transformation tied to the same training-serving pipeline.

Next connection

Class 4 turns from latent components to deliberate feature engineering: creating, encoding, scaling, and organizing variables that better represent the problem.

Course thread: After understanding and compressing data, we make explicit design choices about the features that models and services will consume.

Papers and materials

Jolliffe, Principal Component Analysis; Buitinck et al. (2013), API design for machine learning software; Pedregosa et al. (2011), scikit-learn; Bergstra and Bengio (2012), Random Search.